OBJECT ORIENTED PROGRAMMING THROUGH C++

UNIT 4

1.What is generic programming? Explain how templates support generic programming in C++.

2 What is a function template? Explain with a general example.

3 Explain overloading of template functions in C++ with a suitable example.

4 What is Template and implement of bubble sort.

5 Differentiate between templates and macros in C++.

1What is Generic Programming? Explain how templates support generic programming in C++.

Generic Programming

Generic programming is a programming technique in which algorithms and functions are written independently of the data type.

It allows the same code to work with different data types such as int, float, double, etc.

How Templates Support Generic Programming

C++ supports generic programming using templates.

- A template allows us to write a function or class once.
- The same code can be used for different data types.
- The compiler generates the required version based on the data type.
- It reduces code duplication and improves reusability.

Example:

```
#include <iostream>
using namespace std;

template <class T>
T maximum(T a, T b)
{
```

```
    return (a > b) ? a : b;
}

int main()
{
    cout << maximum(10, 20) << endl;
    cout << maximum(5.5, 2.3);

    return 0;
}
```

Output:

```
20
5.5
```

Conclusion: Templates make C++ programs more flexible, reusable, and type-independent.

---

3. What is a Function Template? Explain with a General Example.

Function Template

A function template is a special type of function that can operate with different data types.

Instead of writing separate functions for int, float, double, etc., we can write one function template.

Important Points:

1. A function template is a generic function.

2. It uses a placeholder such as T to represent a data type.
3. The keyword template is used to declare it.
4. The compiler determines the actual data type when the function is called.
5. It reduces duplicate code.
6. It provides code reusability.
7. It supports different data types using the same function.

General Syntax:

```
template <class T>
T functionName(T a, T b)
{
   // statements
}
```

Example:

```
#include <iostream>
using namespace std;

template <class T>
T add(T a, T b)
{
   return a + b;
}

int main()
```

```
{
    cout << add(10, 20) << endl;
    cout << add(2.5, 3.5);

    return 0;
}
```

Output:

```
30
6
```

Explanation:

- T is a generic data type.
- add() can accept different types of values.
- add(10,20) works with int.
- add(2.5,3.5) works with double.

Conclusion:

A function template allows us to create one general function that can work with multiple data types.

---

4. Explain Overloading of Template Functions in C++ with a Suitable Example.

Template Function Overloading

Template function overloading means defining two or more functions with the same name but different parameter lists.

The compiler selects the appropriate function according to the number or type of arguments passed.

Important Points:

1. Multiple template functions can have the same function name.
2. Their parameter lists must be different.
3. They may differ in the number of parameters.
4. They may also differ in the type of parameters.
5. The compiler determines which function should be called.
6. It provides flexibility and code reusability.
7. It is an example of compile-time polymorphism.

Example:

```
#include <iostream>
using namespace std;

template <class T>
T add(T a, T b)
{
    return a + b;
}

template <class T>
T add(T a, T b, T c)
{
    return a + b + c;
}
```

```
int main()
{
    cout << add(10, 20) << endl;
    cout << add(10, 20, 30);

    return 0;
}
```

Output:

```
30
60
```

Explanation:

- The first add() function accepts two arguments.
- The second add() function accepts three arguments.
- When two values are passed, the first function is selected.
- When three values are passed, the second function is selected.

Conclusion:

Template function overloading allows us to use the same function name for different operations, making programs easier to understand and reuse.

---

5. What is a Template? Implement Bubble Sort using a Template.

Template

A template is a C++ feature used to write generic and reusable code.

It allows the same function or class to work with different data types.

Important Points:

1. Templates support generic programming.
2. They reduce code duplication.
3. They increase code reusability.
4. Templates can be used for functions and classes.
5. The compiler generates the required code according to the data type.
6. Bubble Sort can be implemented using a function template.

Bubble Sort

Bubble Sort is a simple sorting algorithm that repeatedly compares adjacent elements and swaps them if they are in the wrong order.

For example:

5 2 8 1 3

After sorting:

1 2 3 5 8

Bubble Sort using Template:

```
#include <iostream>
using namespace std;

template <class T>
void bubbleSort(T arr[], int n)
{
    for(int i = 0; i < n - 1; i++)
    {
```

```
        for(int j = 0; j < n - i - 1; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                T temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main()
{
    int arr[] = {5, 2, 8, 1, 3};
    int n = 5;

    bubbleSort(arr, n);

    for(int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

Output:

1 2 3 5 8

Advantages:

- The same bubbleSort() can work with different data types.
- It avoids writing separate sorting functions.
- It improves reusability and flexibility.

Conclusion: A template makes Bubble Sort a generic sorting function that can be reused for different data types.

---

6. Differentiate Between Templates and Macros in C++.

Templates vs Macros

Templates	Macros
1. Templates are a feature of the C++ language.	1. Macros are handled by the preprocessor.
2. They are mainly used for generic programming.	2. They are mainly used for text substitution.
3. Templates provide type checking.	3. Macros generally do not provide type checking.
4. Templates are processed by the compiler.	4. Macros are processed before compilation.
5. Templates can be used to create generic functions and classes.	5. Macros are commonly used for constants and code substitution.
6. Templates provide better type safety.	6. Macros can cause unexpected results due to text substitution.

7. Templates provide better code structure and maintainability.	7. Large macros can make programs difficult to understand and debug.
8. Templates use keywords such as template and class.	8. Macros use the #define directive.
9. Templates are generally safer and more suitable for generic programming.	9. Macros are useful for simple preprocessing operations.

Example of Template:

```
template <class T>
T square(T x)
{
    return x * x;
}
```

Example of Macro:

```
#define SQUARE(x) ((x) * (x))
```

Conclusion:

Templates are preferred for generic programming because they provide type safety, reusability, and better program structure, whereas macros perform simple preprocessor-based text substitution.